



TM232

B&R Library Design Guidelines

Requirements

Training modules	TM210 – Working with Automation Studio TM223 – Automation Studio Diagnostics TM24x – At least one programming language
Software	Automation Studio 4 and above
Hardware	none

Table of contents

1 Motivation.....	5
2 Objectives.....	6
3 Reading rules.....	7
4 Rules L001 - L008: Naming conventions.....	8
4.1 Rule L001: Length of library names and function/function block names.....	8
4.2 Rule L002: Naming libraries.....	8
4.3 Rule L003: Naming function blocks.....	8
4.4 Rule L004: Naming constants.....	9
4.5 Rule L005: Naming inputs and outputs.....	9
4.6 Rule L006: Naming structure types.....	9
4.7 Rule L007: Naming enumeration types.....	10
4.8 Rule L008: Naming enumeration elements.....	10
5 Rules L101 - L108: Function blocks.....	11
5.1 Rule L101: "Enable" or "Execute" inputs.....	11
5.2 Rule L102: General structure of the function block interface.....	11
5.3 Rule L103: Reset function blocks with Enable input.....	13
5.4 Rule L104: Optional parameters.....	13
5.5 Rule L105: User structures as inputs.....	13
5.6 Rule L106: Updating parameters.....	14
5.7 Rule L107: Commands.....	14
5.8 Rule L108: Internal structure.....	14
6 Rules L201 - L202: Functions.....	16
6.1 Rule L201: Return value.....	16
6.2 Rule L202: General structure of the function block interface.....	16
7 Rules L301 - L305: Enable/Execute behaviour and status information.....	17
7.1 Rule L301: Standard status information.....	17
7.2 Rule L302: Using Busy - Active.....	18
7.3 Rule L303: Using Enable - Error.....	19
7.4 Rule L304: Using Execute - Done.....	20
7.5 Rule L305: Using Execute - InXxx.....	21
8 Rules L401 - L402: Parameters.....	23
8.1 Rule L401: Parameter check on Enable.....	23
8.2 Rule L402: Parameter check on Update.....	23
9 Rules L501 - L503: User specific commands.....	25
9.1 Rule L501: Command class – Edge-sensitive.....	25
9.2 Rule L502: Command class – Level-sensitive.....	26
9.3 Rule L503: Command class – Edge-sensitive with handshake.....	26
10 Rules L601 - L604: Error handling.....	27
10.1 Rule L601: Error output.....	27
10.2 Rule L602: Using StatusID and ErrorID.....	27
10.3 Rule L603: Using constants for StatusID or ErrorID.....	28
10.4 Rule L604: Unique error identification numbers.....	28

11 Rules L701 - L705: Library practices.....	29
11.1 Rule L701: Null-terminated string.....	29
11.2 Rule L702: Buffer size indication.....	29
11.3 Rule L703: Avoid cycle time violations.....	29
11.4 Rule L704: Memory management.....	29
11.5 Rule L705: Redundancy.....	29
12 Rules L801 - L802: Compatibility.....	31
12.1 Rule L801: Runtime libraries.....	31
12.2 Rule L802: Versioning.....	31

1 Motivation

Software technology in Automation Studio from the user's point of view is largely shaped by libraries. As a result, the terms “technology” and “libraries” are significantly interrelated and sometimes used synonymously.

How these libraries are designed will therefore influence the user's experience when interacting with these products and ultimately justifies the need for shared guidelines.



Guidelines create standards and also help to maintain quality in time-critical situations.

2 Objectives

The objective of these guidelines is to optimize the **quality of the user experience** by ensuring the following:

- Uniform appearance in Automation Studio
- Common and user-friendly operating concept
- Standardized interfaces

In addition, **software quality** should also be improved through the use of:

- A library template with testing environment in accordance with these guidelines
- Defined function block behavior
- Specifications that promote the reusability of code

3 Reading rules

Overview of rules

These guidelines are divided into the following categories:

- Naming conventions
- Function blocks
- Functions
- Enable/Execute behaviour and status information
- Parameters
- User specific commands
- Error handling
- Library practices
- Compatibility

How to read the rules

Each rule can be identified by a "Rule ID" which is a unique 3-digit number.

4 Rules L001 - L008: Naming conventions

This section contains naming conventions for libraries, library elements like functions and function blocks and their interfaces.

4.1 Rule L001: Length of library names and function/function block names

Guidelines

For SG4 targets

- 10 characters can be used for the library name.
- 32 characters can be used for function block names.

4.2 Rule L002: Naming libraries

Libraries should have meaningful names. A prefix may be added to the library name to indicate its inclusion in a specific group of products or technologies.

Examples

Name	Clarification
ArUser	The prefix Ar indicates that the library is part of the Automation Runtime. The name "User" indicates that the library is used to read and write users and roles in the system.
MpRecipe	The prefix Mp indicates that the library refers to a mapp component. This library provides all the functions necessary for recipe management.

Comments

Libraries provided with Automation Studio and the Technology Packages include a prefix.

Mp	Library which is interface to a mapp component.
MT	Library with mechatronic background.
Mc	Library with Motion/CNC/Robotics background.
Ar	Library which is part of the Automation Runtime.
Co	Library which belongs to the technology package mapp Cockpit.
Vi	Library included in technology package mapp Vision.

Prefixes listed in the table above are reserved and may not be used.

4.3 Rule L003: Naming function blocks

Function block names are composed of the full library name and the function name.

Examples

```
MTBasicsDelayTime
MTFilterLowPass
MpRecipeXml
```



```
MpDataRecorder
```

Reasoning

Including the library name in function or function block names makes it easy to identify just by name which library the function or function block belongs to.

4.4 Rule L004: Naming constants

Guidelines

- Name of constants start with the library prefix in lower case. The name that follows is written in uppercase, single words are separated with “_”.
- Constant names are limited to 32 characters.

Constants valid for multiple libraries	Prefix + Descriptive variable name
Constants valid for one specific library	Prefix + Library name + Descriptive variable name

Table 1: Composition of names of constants

Examples

```
mtPOSITIVE_DIRECTION
mpRECIPE_INVALID_FILENAME
```

4.5 Rule L005: Naming inputs and outputs

Input and output variables are written in upper camelcase style.

Examples

Inputs

```
MpLink, ErrorReset, MoveVelocity
```

Outputs

```
CommandBusy, CurrentUser, AccessRights
```

References

TM231 B&R coding guidelines V1.0.0.2: Rule 005: Define the use of case

4.6 Rule L006: Naming structure types

Structure types are written in upper camelcase and end with the suffix Type.

Data types valid for multiple libraries	Prefix + Name + Type
Data types valid for one specific library	Library name + Name + Type
Data types valid for one specific function block	Function block name + Name + Type
Internal function block structures	Function block name + Internal + Type

Table 2: Composition of names of user data types

Examples

```
MtFilterParameterType
MpRecipeXmlHeaderType
```

References

TM231 B&R coding guidelines V1.0.0.2: Rule 007: Use suffixes for user-defined data types

4.7 Rule L007: Naming enumeration types

Enumerated types are written in Upper Camel Casing style and end with the name Enum. They follow the same style defined in [Rule L006: Naming structure types](#)

Examples

```
MTFilterTypeEnum  
MTBasicsPWMModeEnum  
MpRecipeXmlLoadEnum
```

References

TM231 B&R coding guidelines V1.0.0.2: Rule 007: Use suffixes for user-defined data types

4.8 Rule L008: Naming enumeration elements

Guidelines

- Names of enumerated type elements start with the library prefix in lower case followed by a meaningful name written in UPPER_SNAKE_CASE.
- These elements are limited to 32 characters.

Examples

```
mtPULSE_MIDDLE  
mpRECIPE_XML_LOAD_HEADER
```

5 Rules L101 - L108: Function blocks

Function blocks are characterized by the following:

Enable/Execute input

The inputs Enable and Execute are used to start the function block. These commands work together with defined status information outputs. The exact behavior of these variables and how they are best used is described in section [Rules L301 - L305: Enable/Execute behaviour and status information](#)

Internal memory

Function blocks can have internal memory which also has an impact on the outputs, see [Rule L108: Internal structure](#)

More than one return value

Function blocks can have more than one return value.

5.1 Rule L101: "Enable" or "Execute" inputs

To start a function block, either an "Enable" input or "Execute" input is used.

Reasoning

Function blocks are classified into two groups:

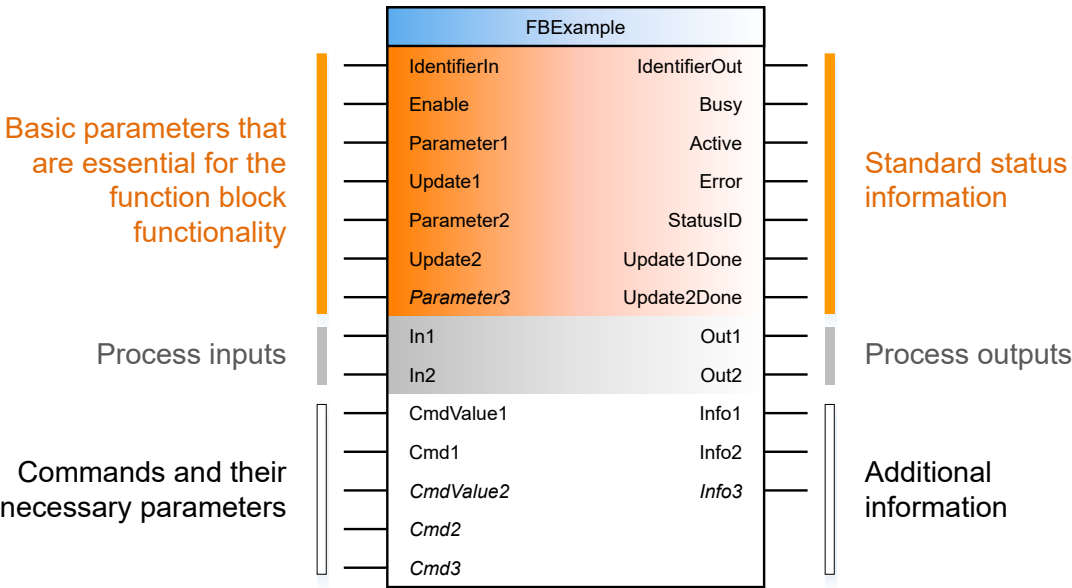
- Edge Triggered Function Blocks – which are coupled to the "Execute" input
- Level Controlled Function Blocks – which are coupled to the "Enable" input

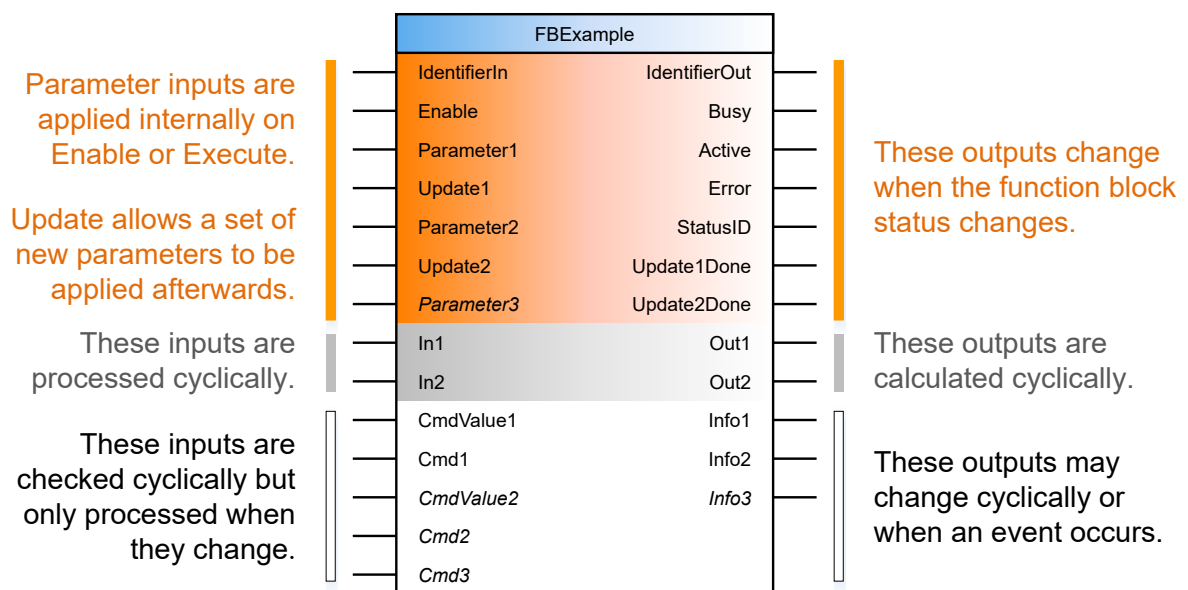
More details about these two inputs are available in [Rules L301 - L305: Enable/Execute behaviour and status information](#)

References

PLCopen Software Creation Guidelines: Creating PLCopen Compliant Libraries Version 1.0. Chapter 3 Introduction of the PLCopen Function Block concepts

5.2 Rule L102: General structure of the function block interface





A function block is divided into three parts:

Upper part (orange)

- Commands used to start the function block like Enable or Execute are used here.
- Other commands that affect the general behaviour of the function block like Update and ErrorReset are permitted to be used here.
- Only the standard status outputs ([Rule L301: Standard status information](#)), StatusID and UpdateDone are permitted to be used here.
- The identifier, if necessary, is always above Enable or Execute.
- A parameter structure is always located above the corresponding update command.

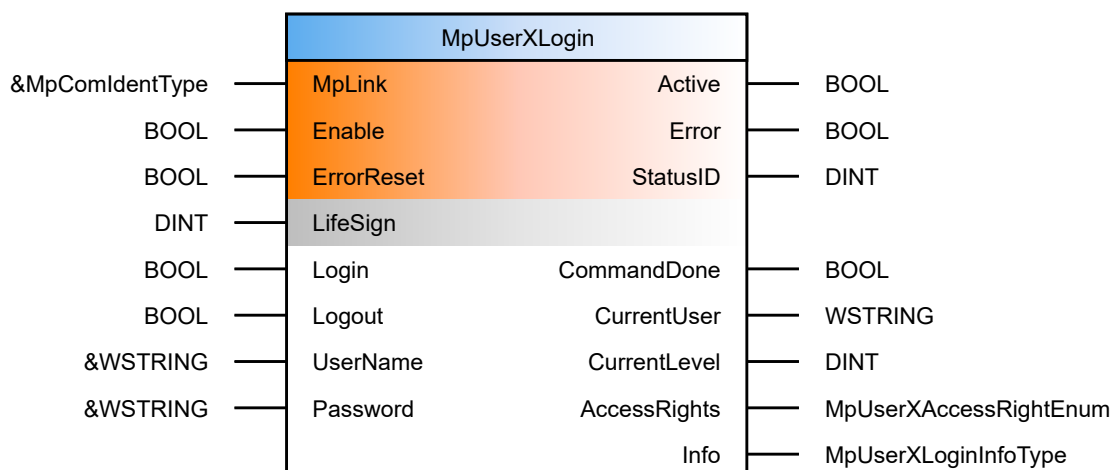
Middle part (gray)

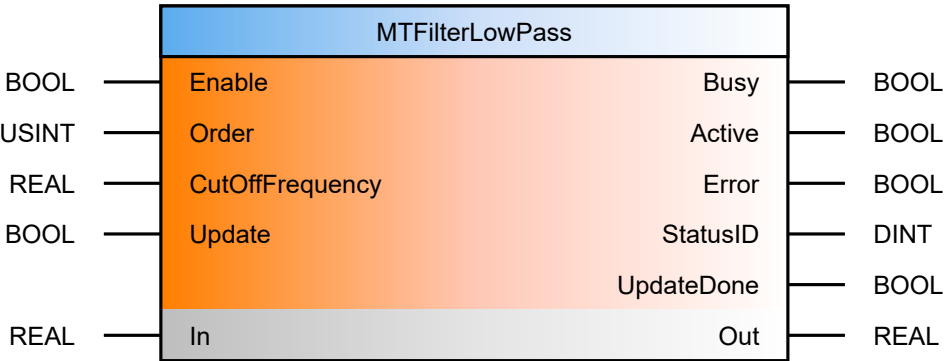
- Consists of inputs and outputs which are cyclically processed. (compared to the "Update" command and the commands in the white section, which are only executed upon positive edges).

Lower part (white)

- Contains additional commands accompanied by the value the command needs. These values are placed directly above the command input.
- The output variables provide additional useful information.

Examples





5.3 Rule L103: Reset function blocks with Enable input

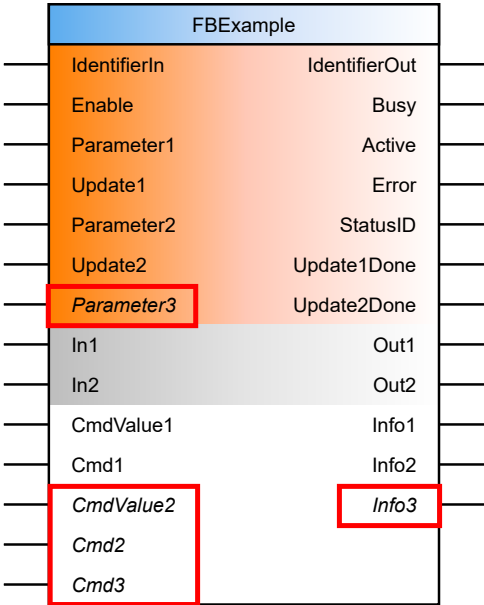
When the status FALSE is detected on the input variable Enable, the internal state and the output variables are reinitialized and the function block is ready for a new start.

References

PLCopen Software Creation Guidelines: Creating PLCopen Compliant Libraries Version 1.0. Chapter 3 Introduction of the PLCopen Function Block concepts

5.4 Rule L104: Optional parameters

Optional parameters are interface parameters for the function block. Even if these parameters are not set, the function block should work without error.



Comments

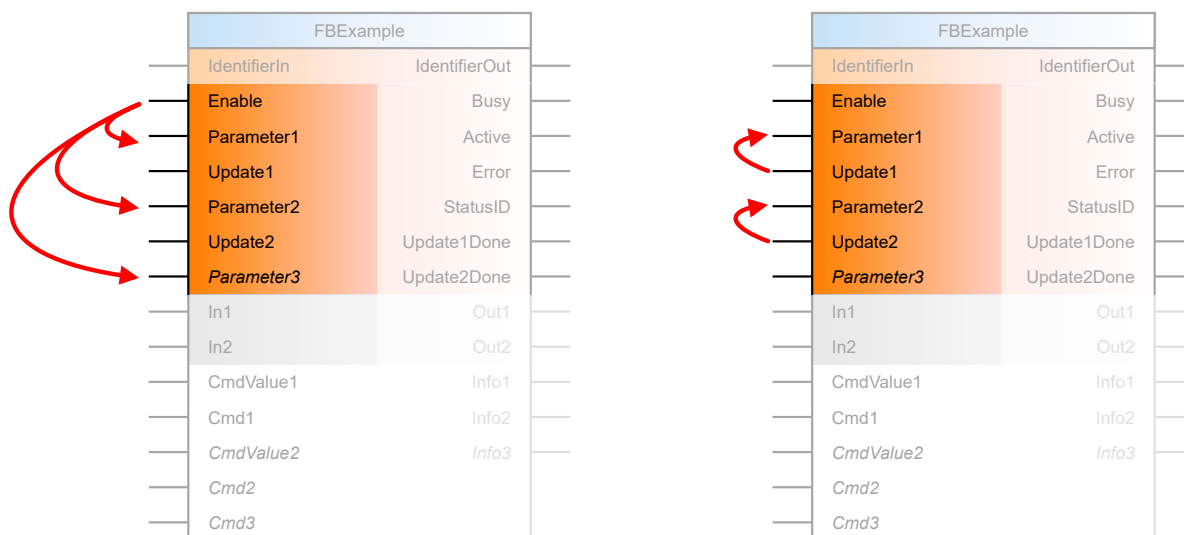
In Automation Help, these parameters are usually indicated in italics.

5.5 Rule L105: User structures as inputs

- Guidelines**
- Direct integration of structures into the function block interface is preferred. If necessary, pointers to structures are also permitted.
 - Parameters should be combined into one structure if it improves usability.

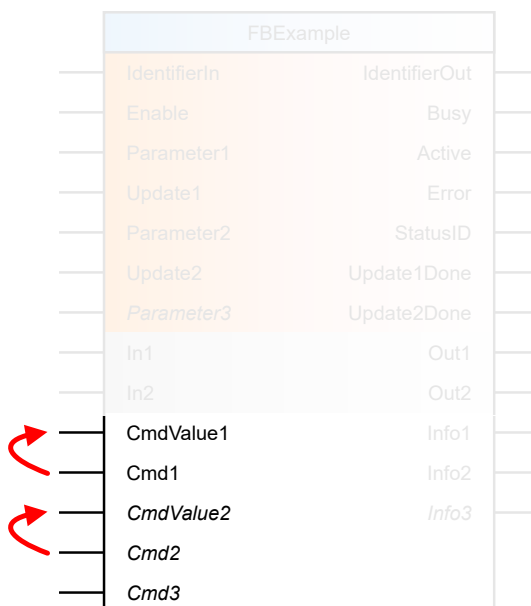
5.6 Rule L106: Updating parameters

When Enable or Execute is set, all parameters in the orange area are taken into account. Update only acts on the parameters directly above it. Changes in the Parameters should only take effect when either Enable or Execute or Update are set.



5.7 Rule L107: Commands

A command is accompanied by the values it needs. The values are written above the command input.



5.8 Rule L108: Internal structure

Every function block has a variable called "Internal". It contains data needed while the function block is active.

The structure name is defined according to [Rule L006: Naming structure types](#). Variable within the structure follow the naming convention in “Rule 005: Define the use of case” from “TM231 B&R Coding guidelines”.

FB	MTFilterLowPass	
Enable	BOOL	
Type	MTFilterTypeEnum	
Order	USINT	
CutOffFrequency	REAL	
Update	BOOL	
In	REAL	
Busy	BOOL	
Active	BOOL	
Error	BOOL	
StatusID	DINT	
UpdateDone	BOOL	
Out	REAL	
SystemReference	UDINT	
Internal	MTFilterLowPassInternalType	

6 Rules L201 - L202: Functions

Functions are characterized by the following:

No Enable/Execute input

Functions are executed every function call.

No internal memory

If information should be stored for the next function call, then a function block must be used instead.

Only one return value

If more than one value should be returned, e.g. result and error code, then a function block must be used instead.

6.1 Rule L201: Return value

Guidelines

- If functions have no return value they should return StatusID indicating the status of the operation.
- If functions have a useful return value, they do not provide StatusID. An error is indicated by a suitable return value.

6.2 Rule L202: General structure of the function block interface

Guidelines

- Functions only have one orange area.
- ReturnValue is the only output variable.

7 Rules L301 - L305: Enable/Execute behaviour and status information

Behavior of Enable

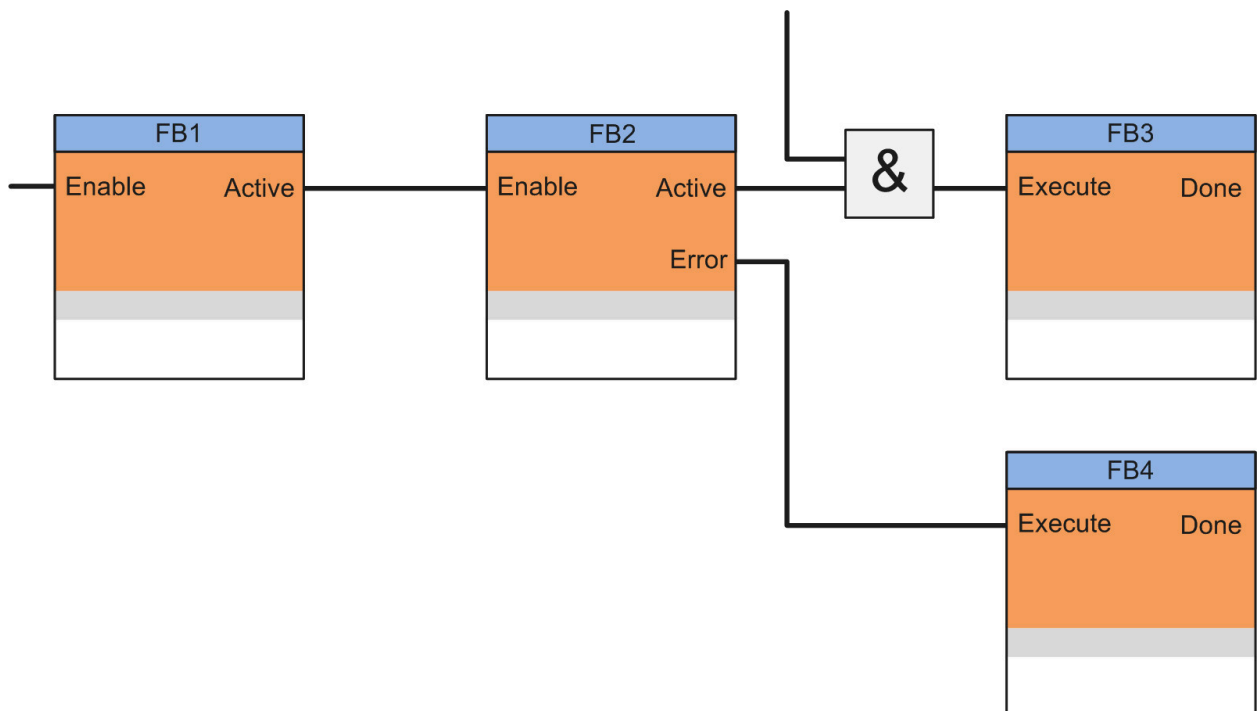
- The Enable command is level sensitive.
- This functionality should be used to switch the function block on and off like a power supply. When it is switched on (enabled), additional commands should be used to perform some action.
- It should not be used to stop the FB execution temporarily, e.g. when you want to pause a control loop. The Hold command should be used instead.
- Status outputs like Active show that the function block is enabled, initialized and ready to perform its task.

Behavior of Execute

- The Execute command is edge sensitive.
- This functionality should be used to trigger a specific function or process. The function block is only executed as long as this process is active.
- For the user it is important to know when the process ends. Therefore, status outputs like Done are used to show this event.
- When the name of a function block sounds like a command, it's probably a function block with Execute. For example, FBExampleCreateFile.

Why this approach makes sense

The advantage is that applications become easier to program while allowing a precise management of command sequences to control machine actions.



7.1 Rule L301: Standard status information

Guidelines

- Standard status outputs are always related to an Enable or Execute input. They are therefore located in the upper part (orange) of the function block interface (Rule L102: General structure of the function block interface).
- They have a defined hierarchy and functionality.
- Each status output exists as BOOL.

Depending on the function block type the following standard status outputs must be used:

Status output	Enable	Execute	Description
Done			Set when the command action has been successfully completed.
InXxx			This output can be used instead of Done. InXxx stands for InVelocity, InPosition, InTorque, InSync, etc. It is mutually exclusive to Done (only one can be present) but has different behavior.
Busy			The function block is working. This output remains High as long as the function block is performing any action.
Active			Initialization is done. The function block inputs are controlling the process.
CommandAborted			The function block action started with Enable or Execute is interrupted by another function block.
Error			An error has occurred and an error code is indicated on StatusID/ErrorID. The function block is in error state.

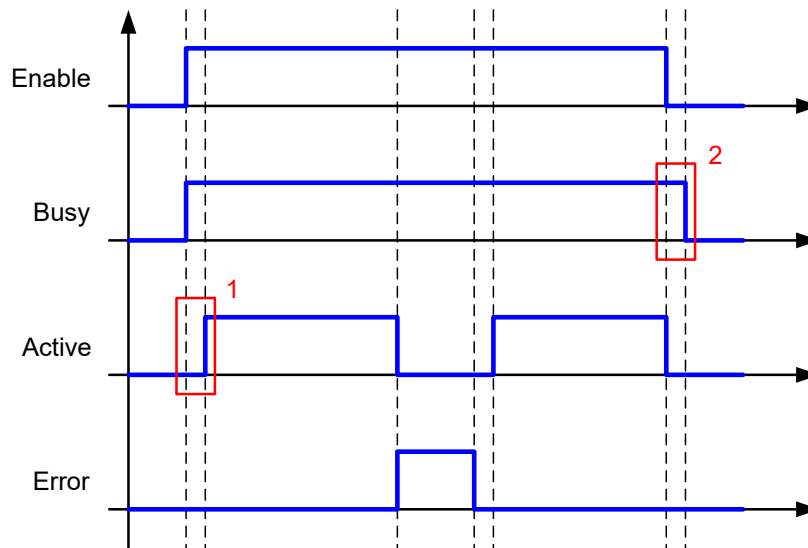
 Mandatory
  Optional
  Not permitted

It is important that this order from top to bottom be maintained on the function block outputs.

7.2 Rule L302: Using Busy - Active

Busy - Active combination

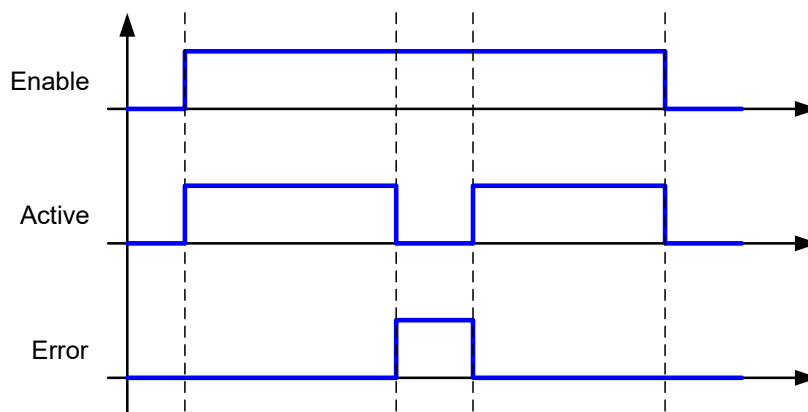
A Busy–Active combination makes sense for example when the function block is working together with hardware and time is needed to initialize and shut down this device. Another case is when an asynchronous process is executed in the background for initialization purposes.



1...Initialization 2...Exit

Active without Busy

It is also possible to use Active without Busy. One example would be a simple function block running on the PLC without hardware dependencies and not waiting for asynchronous processes in the background, e.g. MTFILTERLowPass.

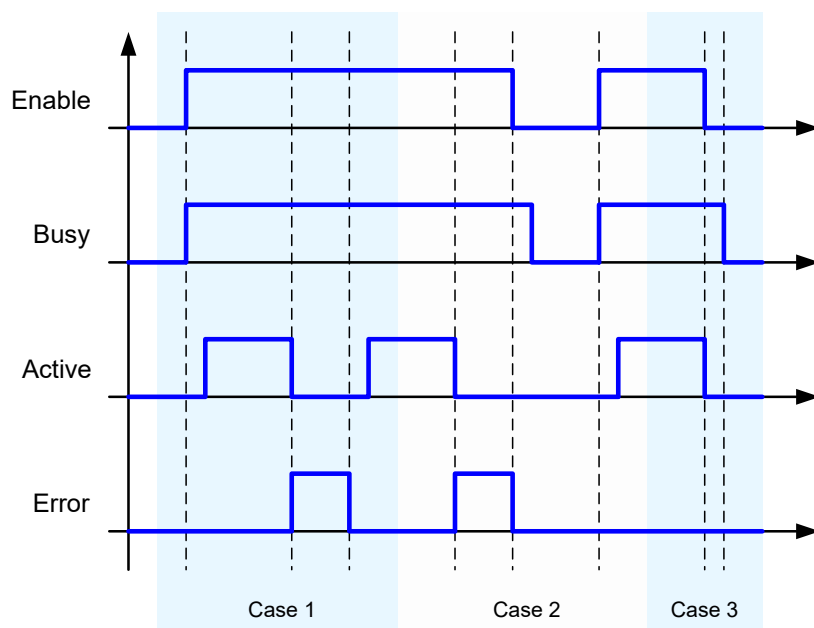


No initialization or exit subroutine is necessary here.

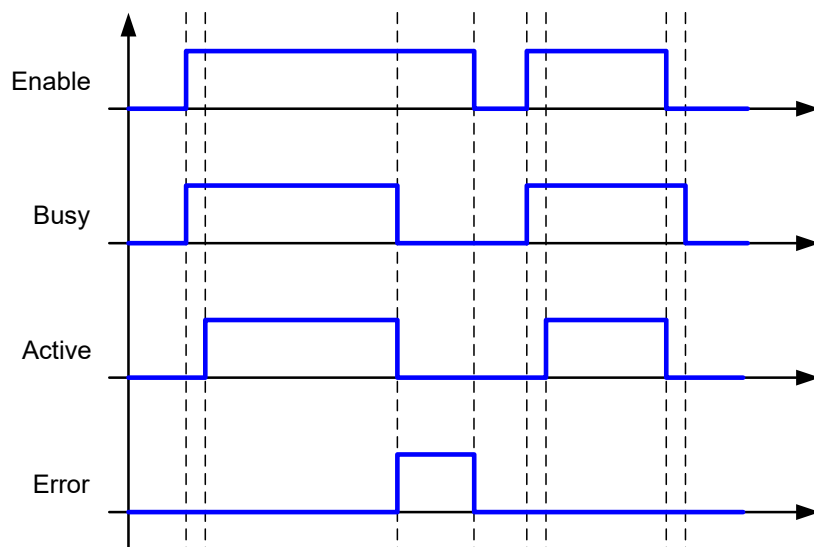
7.3 Rule L303: Using Enable - Error

Errors that can be cleared automatically

- Case1: Error cleared automatically
- Case2: Disable procedure with active error
- Case3: Disable procedure when there is no error



Errors that cannot be cleared automatically



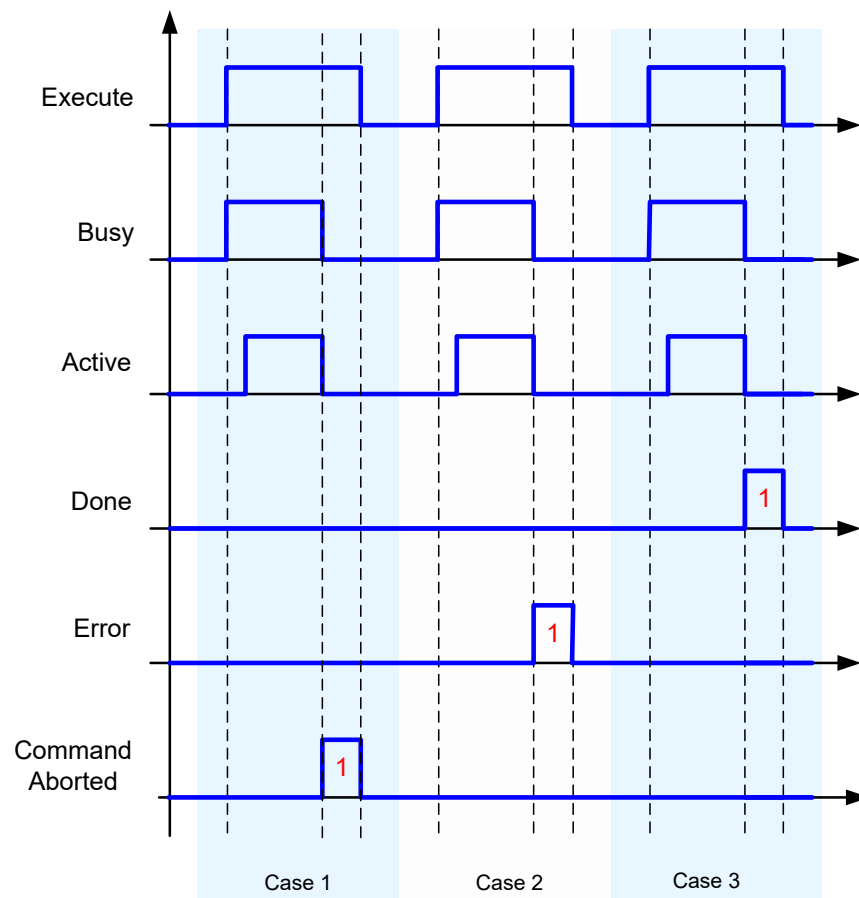
7.4 Rule L304: Using Execute - Done

Guidelines

- Outputs Busy (or Active) - Done - Error - CommandAborted are mutually exclusive. This means only one of them can be High.
- In application code, Done can be used to reset Execute. This kind of handshake is frequently used.
- If Done/Error/CommandAborted are reset to zero the function block must return to its initial state. This applies to all outputs and variables of the internal structure.

Scenarios

- Case1: Function block execution aborted
- Case2: Error occurs
- Case3: Function block finished successfully



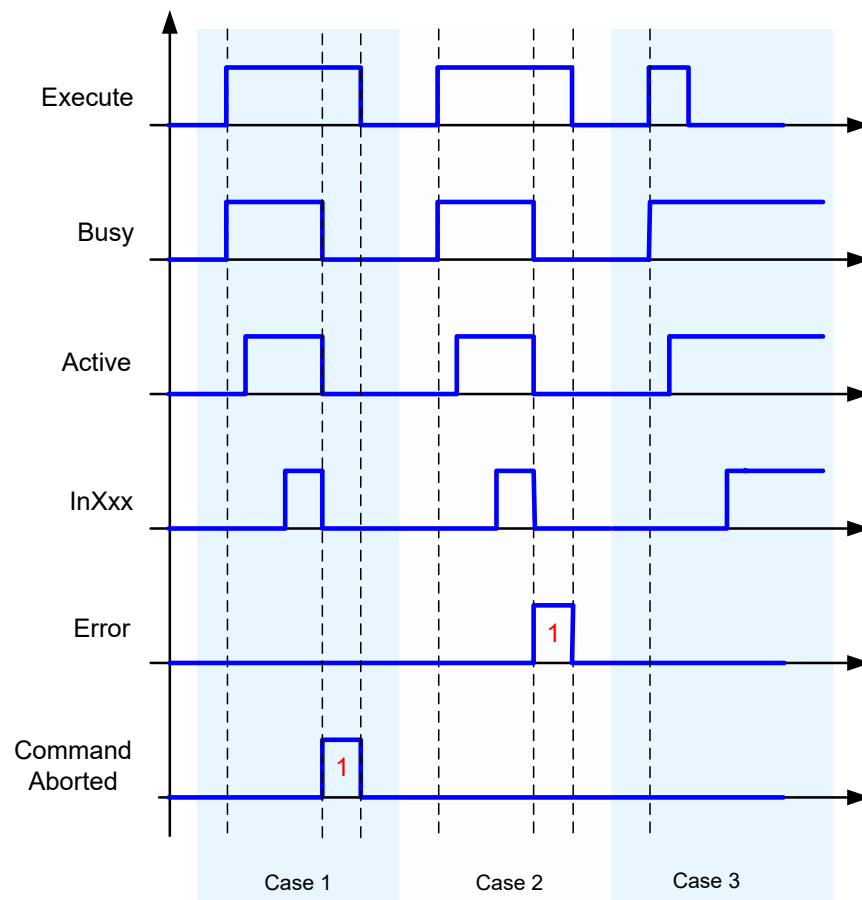
1...If Execute is reset before these outputs are set, they remain high only for one cycle.

7.5 Rule L305: Using Execute - InXxx

Outputs Busy (or Active) - InXxx - Error - CommandAborted are mutually exclusive. This means only one of them can be High.

Scenarios

- Case1: Function block execution aborted
- Case2: An error occurs
- Case3: Function block is working without errors



1...If Execute is reset before these outputs are set, they remain High only for one cycle.

8 Rules L401 - L402: Parameters

Definition

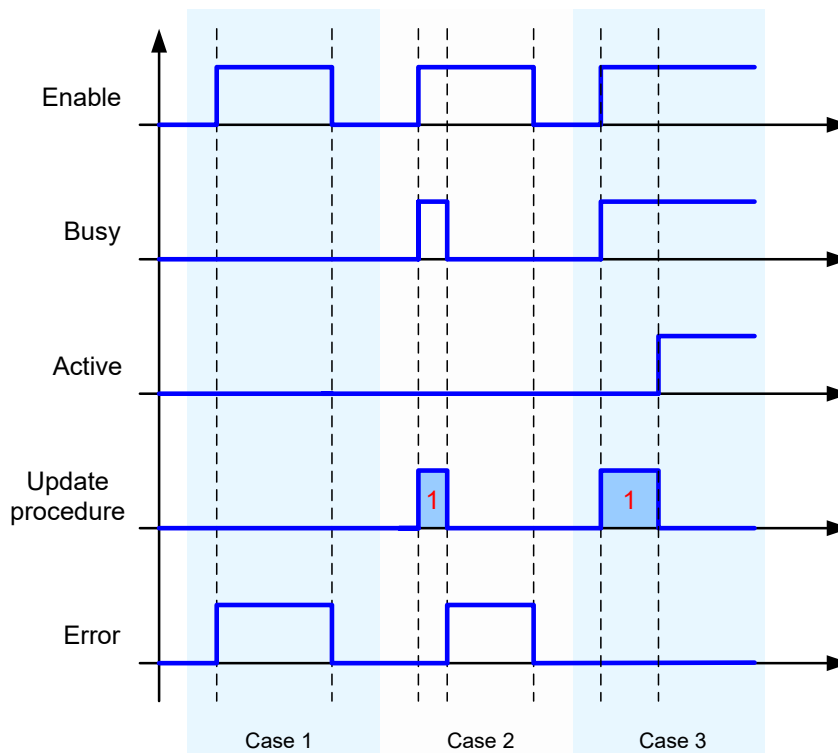
- Parameters are used to identify a characteristic, feature, measurable factor that helps to define a system.
- Parameters are constant values. They are only applied internally when confirmed by a specific command like Enable, Update or SetOut.

8.1 Rule L401: Parameter check on Enable

An error during an update procedure together with command Enable is treated like an error that cannot be cleared automatically ([Rule L303: Using Enable - Error](#)).

Likely scenarios

- Case1: Parameter check failures immediately when Enable is set
- Case2: Error during the parameter update procedure
- Case3: Parameter check successful



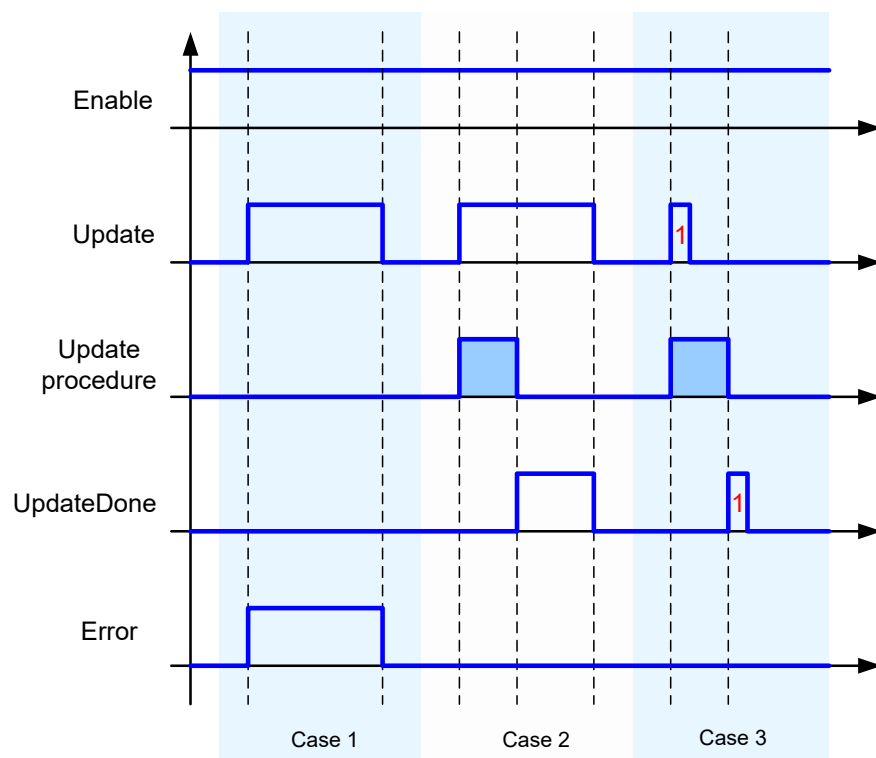
1.. Sometimes the update procedure takes more than one cycle, especially, when hardware or asynchronous functions are involved.

8.2 Rule L402: Parameter check on Update

An error during an update procedure together with the command Enable is treated like an error that cannot be automatically cleared ([Rule L303: Using Enable - Error](#)).

Likely scenarios

- Case1: Parameter check failure immediately when Update is set
- Case2: Parameter check successful – Update command reset after the update procedure has completed. In application code, UpdateDone can be used to reset Update. This kind of handshake is used frequently.
- Case3: Parameter check successful – Update command reset immediately

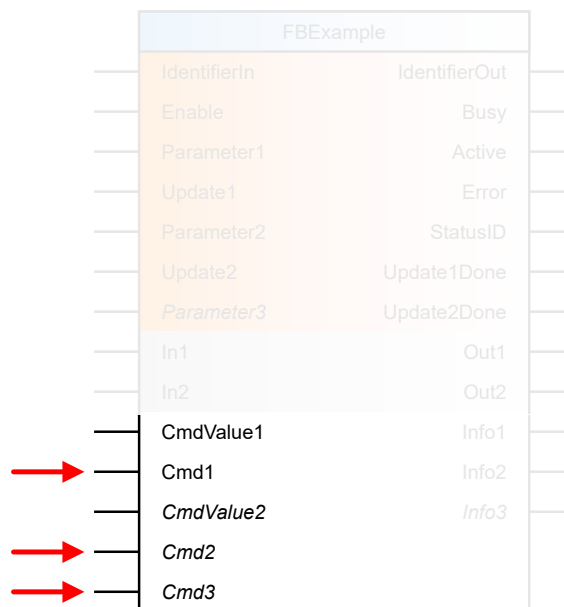


1...If Update is reset before UpdateDone is set, it remains High only for one cycle.

9 Rules L501 - L503: User specific commands

User specific commands may appear in the white part of a function block. To make sure these commands have the same functionality three command classes are defined in this section:

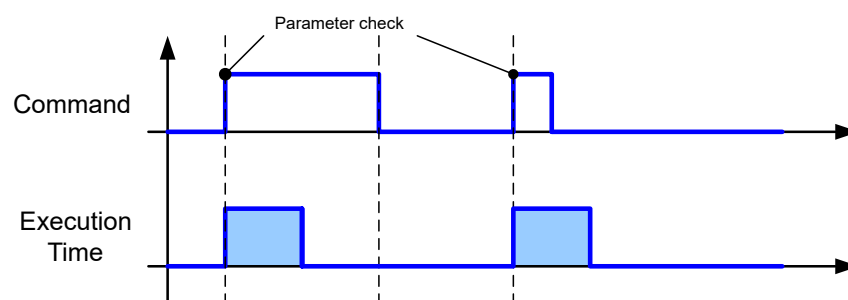
- Command class – edge sensitive
- Command class – level sensitive
- Command class – edge sensitive with handshake



The following pool of command names are associated with frequently used functions should be used.

- Power
- Hold
- Home
- Reset
- Stop
- QuickStop

9.1 Rule L501: Command class – Edge-sensitive



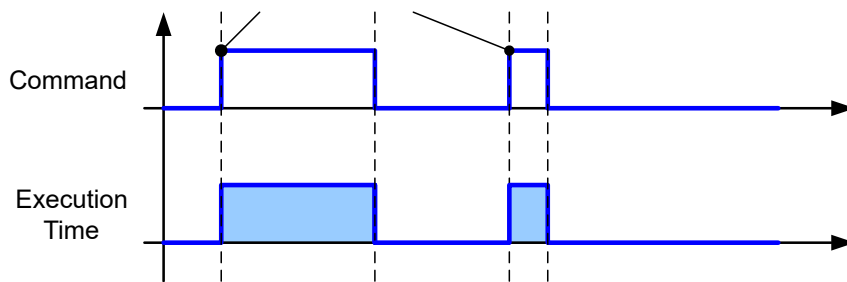
Typical applications

- Action with fixed execution time.
- No feedback necessary that command execution has finished.

Examples

Setting or resetting an output variable

9.2 Rule L502: Command class – Level-sensitive

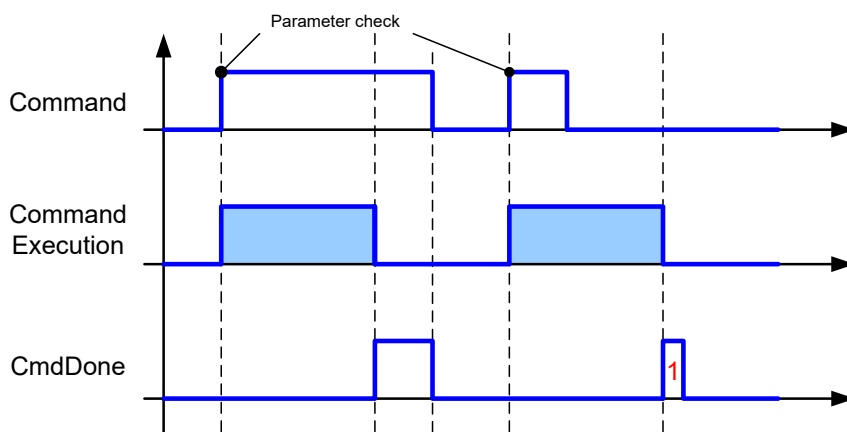
**Typical applications**

- Action started and stopped with the same command.

Examples

Keeping the output variable constant as long as the command is set.

9.3 Rule L503: Command class – Edge-sensitive with handshake



1...If Command is reset before CmdDone is set, it remains High only for one cycle.

Typical applications

- Action started with a specific command. Maybe its duration is unknown and it is important to know when its execution has finished successfully.

Examples

Starting a tuning procedure.

Sev	Severity code • 00 = Success • 01 = Informational • 10 = Warning • 11 = Error
C	Customer • 0 = B&R • 1 = Customer
R	Reserve
Facility	Indicates the area this error number belongs to
Code	Error code

Examples

Constants valid for more libraries	mtERR_INVALID_PARAMETER
Constants valid for one certain library	mpRECIPE_ERR_INVALID_FILENAME
	mtBASICS_WRN_OUTPUT_IN_SATURATION

10.3 Rule L603: Using constants for StatusID or ErrorID

Constants should be used to represent error or status numbers. These constants follow the naming convention mentioned in [Rule L004: Naming constants](#)

The following scheme of abbreviations should be used:

- Prefix: lowercase
- Library name: UPPERCASE
- Descriptive variable name: UPPER_SNAKE CASE
- ERR: for constants which represent an error number
- WRN: for constants which represent a warning
- INF: for constants which share information

Constants valid for more libraries	Prefix + ERR_descriptive variable name Prefix + WRN_descriptive variable name Prefix + INF_descriptive variable name
Constants valid for one certain library	Prefix + library name + _ERR_descriptive variable name Prefix + library name + _WRN_descriptive variable name Prefix + library name + _INF_descriptive variable name

Examples

Constants valid for more libraries	mtERR_INVALID_PARAMETER
Constants valid for one certain library	mpRECIPE_ERR_INVALID_FILENAME
	mtBASICS_WRN_OUTPUT_IN_SATURATION

10.4 Rule L604: Unique error identification numbers

Each library has its own set of error identification numbers. Which means it is not allowed to write error numbers of internally used function blocks of other libraries to StatusID or ErrorID.

11 Rules L701 - L705: Library practices

This section contains guidelines for "good" library development practices.

11.1 Rule L701: Null-terminated string

Each string specified must be a null-terminated string.

11.2 Rule L702: Buffer size indication

If a string should be written to a specified buffer and the buffer size is too small, an additional output variable beside ErrorID must indicate the necessary buffer size.

11.3 Rule L703: Avoid cycle time violations

To avoid cycle time violations, function calls that may lock the system are not permitted.

Examples

Function calls that may lock the system:

- Usage of device open or malloc

11.4 Rule L704: Memory management

Allocating and deallocating memory in the cyclic section of a function block is not permitted since it causes memory fragmentation.

- Allocating memory in the initialization section of a library
- Providing it as an input buffer that is specified by the user

If memory is provided by the user, its size must also be specified as an input value.

11.5 Rule L705: Redundancy

Each function block must be designed for use in redundant systems.

Labels

The label indicating whether a PV should be synchronized or if a library or function block is capable of redundancy is done using pragmas in the .fun file. The following pragmas are available:

Pragma	Description
REDUND_REPLICABLE	Labels variables, structures or function blocks that will be synchronized.
REDUND_UNREPLICABLE	Labels variables, structures or function blocks that will not be synchronized.
REDUND_OK	Labels function blocks or functions for unrestricted use in redundant configurations.
REDUND_CONTEXT	Labels function blocks or functions for restricted use in redundant areas. Behavior may differ on the active and inactive controller depending on the operating phase.
REDUND_ERROR	Labels function blocks or functions that cannot be used in a redundant configuration.

In essence, only variables that are not being synchronized must be labeled.

Examples

The interface and functionality of the function blocks are not affected by the change

```
FUNCTION_BLOCK FBExample1
    VAR_INPUT
        VarIn01 : {REDUND_REPLICABLE} INT;
        VarIn02 : {REDUND_UNREPLICABLE} USINT;
        VarIn03 : USINT;
    END_VAR
    VAR_OUTPUT
        VarOut01 : {REDUND_UNREPLICABLE} USINT;
        Varout02 : USINT;
    END_VAR
    VAR_IN_OUT
        VarInOut01 : INT;
        VarInOut02 : {REDUND_UNREPLICABLE} USINT;
        VarInOut03 : USINT;
    END_VAR
    VAR
        Var01 : {REDUND_UNREPLICABLE} USINT;
        Var02 : USINT;
    END_VAR
END_FUNCTION_BLOCK

{REDUND_OK} FUNCTION_BLOCK FBExample2
    VAR_INPUT
        Enable : BOOL;
        Device : {REDUND_UNREPLICABLE} UDINT; (* pointer *)
        IPAddress : {REDUND_UNREPLICABLE} UDINT; (* pointer *)
        Length : USINT;
    END_VAR
    VAR_OUTPUT
        StatusID : DINT;
    END_VAR
    VAR
        State : UINT; (* internal variable *)
    END_VAR
END_FUNCTION_BLOCK
```

Data tracking

In principle, data tracking is only necessary for function blocks. Functions have no memory and can therefore be ignored in this regard.

Data not to be tracked

- PV elements that are only relevant locally (e.g. pointers, handles, IDs, etc) are not permitted to be tracked at all.
- These elements are not permitted to be tracked in connection with function blocks that return data as a result or as a temporary value that can or should be different on the two controllers. One such example of this would be the amount of available memory.

Context based redundancy capability

- Functions or function blocks that are not automatically capable of redundancy may still be capable of redundancy based on context. In principle, this applies to functions that could provide different results on the two controllers within a specific system tick. The user must ensure that hot redundancy is maintained on the system when using these function blocks. They are labeled with the pragma {REDUND_CONTEXT}.

Active changes

- If it only makes sense to execute a function or function block on the active controller in a redundant system, then a corresponding query must be added to the source code.

Not capable of redundancy

- Any function or function block whose enabling in the context of a redundant system leads to a loss of synchronization between the controllers must be forbidden. This is done by labeling the function or function block as not capable of redundancy using the pragma {REDUND_ERROR}. An example of this would be functions that change the system configuration.

12 Rules L801 - L802: Compatibility

Function block interface

The function block interface is defined by its input and output variables as well as by the variables of the internal structure (Rule L108: Internal structure).

Changes to an existing library can be classified as follows:

Compatible changes

The interface and functionality of the function blocks are not affected by the change

Examples

- Adding functions and/or function blocks to the library
- Adding data types or constants

Incompatible changes

The interface or functionality of the function blocks is affected by the change.

Incompatible changes may require the application to be modified, which may take a considerably longer amount of time.

Examples

- Changing the name of the following:
 - Function or function block
 - Input or output parameters
 - Data types or constants
- Changing the value of a constant
- Any changes to internal variables

12.1 Rule L801: Runtime libraries

Only compatible changes are permitted to be made to Automation Runtime specific libraries.

12.2 Rule L802: Versioning

Incompatible changes can be made to B&R standard libraries. However, any such changes must be clearly communicated to the user. One possible way to indicate an incompatible library change is by adjusting the version number accordingly.

V 0.xx.x	Versions prior to first official release.
V x.xx.9	Test version, only for unofficial test purposes.
V 0.xx.X	Guaranteed compatible changes, small bug fixes.
V 0.xX.x	Guaranteed compatible changes, small bug fixes, functional expansions.
V 0.Xx.x	Changes may be incompatible. Please, read the release information.
V X.xx.x	Changes may be incompatible. Please, read the release information.

Automation Academy

Your knowledge advantage

The Automation Academy provides **targeted training** courses for our customers as well as for our own employees. Expand your skills in the field of automation technology and learn to independently implement **efficient automation solutions** using B&R systems.

Decide for yourself which **learning concept** you prefer!



Classroom Learning



Virtual Classroom
Learning



Online courses

Classroom learning

B&R offers **standard seminars** at all B&R locations. Services include seminar documents, effective communication of learning content by experienced trainers and an Automation Diploma. A combination of group work and self-study provides the high level of flexibility needed to maximize the learning experience.

Virtual classroom

Remote Lectures supplement B&R's continuing education portfolio with a virtual classroom, offering an alternative to our on-site seminars. Selected content from our standard seminars is offered online. In addition to remote learning methods, powerful simulation tools and secure remote maintenance are used.

Online courses

Take control of the content and learn at your own pace. With B&R **online courses**, you can take your first steps in the world of B&R automation at any time. Based on a comprehensive narrative, you will independently work out how to use our products. The mix of different media allows a logical sequence to be followed when learning as well as a selective choice of information to be used as a reference tool.

Contact

Would you like additional training? Are you interested in finding out what the B&R Automation Academy has to offer? If so, this is the right place.

Access additional information here:

<https://www.br-automation.com/de/academy/>

Enjoy your next training course!



AutomationAcademy





B&R
Industrial Automation GmbH
A member of the ABB Group
B&R Straße 1
5142 Eggelsberg, Austria
office@br-automation.com

t +43 7748 6586-0
f +43 7748 6586-26

br-automation.com

